

ULK POLYTECHNIC INSTITUTE
Cryptography & Network Security (ETTCS801)
Integrated Situation – Final Technical Report

Student name: MUGISHO MWIMUKA

Student ID: 4202670061

Module Code: ETTCS801

Lecturer: Isaac TUMWINE

September 29, 2026

1 Introduction & Project Overview

This report presents the security assessment, implementation, and traffic filtering configuration carried out for the institute's infrastructure. To address weak passwords, unencrypted data transfers, outdated software and unauthorized network access, a three-tier defense strategy was applied:

1. A formal risk assessment prioritizing critical assets and vulnerabilities.
2. A Python security toolkit implementing symmetric encryption (AES via Fernet) and SHA256 hashing for integrity verification.
3. Host-based packet filtering with iptables on Linux to restrict server access according to network subnets.

2 Risk Assessment

2.1 Asset and Vulnerability Identification

- Asset 1: Student records (database/files)
 - *Vulnerability*: unencrypted storage and unencrypted transfers between campuses.
 - *Consequence*: interception, tampering or exposure of grades and personal data.
- Asset 2: Central server
 - *Vulnerability*: Guest network can reach the server without firewall isolation.
 - *Consequence*: network discovery, lateral movement or DoS from untrusted networks.
- Asset 3: Administrator / staff accounts
 - *Vulnerability*: weak passwords and outdated software.
 - *Consequence*: brute-forcing, administrative compromise, privilege escalation.

Risk	Likelihood	Impact	Priority
Unencrypted file storage & transfer	High	High	Critical (Rank 1)
Guest network access to central server	High	Medium	High (Rank 2)
Weak passwords & outdated software	Medium	High	Medium (Rank 3)

Table 1: Risk ranking matrix

2.2 Risk Evaluation and Ranking

- Rank 1: student records travel unencrypted over shared links; interception causes loss of confidentiality and reputational damage.
- Rank 2: Guest devices can scan and probe open server ports because no boundary firewall separates them.
- Rank 3: the impact is severe, but exploitation requires active reconnaissance or brute force, so it is less immediate than open routes and cleartext transfers.

2.3 Recommended Security Controls

1. Data security: symmetric encryption (AES) for stored records and HTTPS/SFTP for all cross-campus transfers.
2. Network filtering: iptables rules dropping traffic from the Guest subnet (192.168.20.0/24).
3. Access control: strong password policy, multi-factor authentication where possible, and regular patching.

3 Encryption and Integrity Application

3.1 Implementation Overview

The utility `security_toolkit.py` offers command-line encryption, decryption, hashing and integrity verification using the cryptography library and the Python standard library:

- Symmetric encryption: Fernet (AES-128 in CBC mode with HMAC authentication).
- Integrity hashing: `hashlib.sha256()` produces a 256-bit digest before and after transfer.
- Error handling: missing files and invalid or corrupted ciphertexts are caught to avoid crashes.

3.2 Usage

```
python3 security_toolkit.py --encrypt FILE python3 security_toolkit.py --decrypt FILE.enc
python3 security_toolkit.py --hash FILE python3 security_toolkit.py --verify FILE
EXPECTED_SHA256
```

3.3 Source Code (main functions)

```

import hashlib, os, sys
from cryptography.fernet import Fernet KEY_FILE = "secret.key"

def generate_or_load_key():
    """Generate a Fernet key or load the existing one.""" if not
    os.path.exists(KEY_FILE): key = Fernet.generate_key() with open(KEY_FILE, "wb")
    as kf:
        kf.write(key)
    else:
        with open(KEY_FILE, "rb") as kf:
            key = kf.read() return key

def calculate_sha256(file_path): sha256_hash =
    hashlib.sha256() with open(file_path, "rb") as f:
        for block in iter(lambda: f.read(4096), b''):
            sha256_hash.update(block)
    return sha256_hash.hexdigest()

def encrypt_file(file_path):
    fernet = Fernet(generate_or_load_key()) with open(file_path,
    "rb") as f:
        data = f.read()
    with open(file_path + ".enc", "wb") as f:
        f.write(fernet.encrypt(data))

def decrypt_file(file_path):
    fernet = Fernet(generate_or_load_key()) with open(file_path,
    "rb") as f:
        encrypted = f.read()
    try:
        return fernet.decrypt(encrypted)
    except Exception as e:
        print(f"[!] Decryption failed: {e}") sys.exit(1)

```

3.4 Test Evidence: Encryption, Decryption and Tamper Detection Paste below the *real* output of your terminal after running the commands above on a sample file.

```

$ python3 security_toolkit.py --encrypt sample_student_record.txt
<< paste your real output here >>

$ python3 security_toolkit.py --decrypt sample_student_record.txt.enc
<< paste your real output here >>

$ python3 security_toolkit.py --verify tampered_student_record.txt < hash>
<< paste your real output here >>

```

4 Network Traffic Filtering Configuration & Testing

4.1 Firewall Rule Architecture

The central server (192.168.10.100) uses a default-deny ingress policy: Staff traffic (192.168.10.0/24) is allowed on port 80 only, Guest traffic (192.168.20.0/24) is dropped, and everything else is blocked.

```
sudo iptables -F sudo iptables -P INPUT DROP sudo iptables -P FORWARD DROP sudo iptables -A INPUT -i lo -j ACCEPT sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT sudo iptables -A INPUT -s 192.168.20.0/24 -j DROP sudo iptables -A INPUT -s 192.168.10.0/24 -p tcp --dport 80 -j ACCEPT sudo iptables -A INPUT -p tcp --dport 80 -j DROP
```

4.2 Test Execution

Tests were run with netcat (one permitted connection, two blocked).

Test	Command	Result
1. Staff, port 80 (permitted)	nc -zv -s 192.168.10.15 192.168.10.100 80	Connection succeeded – PASS
2. Guest subnet (blocked)	nc -zv -w 3 -s 192.168.20.45 192.168.10.100 80	Timed out (silent drop) – PASS
3. Staff, port 22 (blocked)	nc -zv -w 3 -s 192.168.10.15 192.168.10.100 22	Timed out (default DROP) – PASS

Table 2: Firewall test results

5 Conclusion

The measures address the vulnerabilities found in the risk assessment. Data at rest and in transit are protected by symmetric encryption and SHA-256 integrity checks, while the iptables policy isolates the central server from the Guest subnet and from unauthorized service ports.

GitHub repository:

<https://github.com/Romuald-Mw/named-cryptography-network-security-exam>.